

u2110409

March 22, 2024

1 Question 1

We can model the resultant force in terms of the states $\mathbf{q}$ and $\mathbf{p}$ and their derivatives. Since $f = ma$:

$$m\ddot{\mathbf{q}} = -\gamma \left(\dot{\mathbf{q}} - \beta \begin{bmatrix} -\frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \end{bmatrix} \mathbf{q} \right) - \mathbf{u}$$

Since the desired result is a 1st order state-space model, $\ddot{\mathbf{q}}$ needs to be converted to something different.

The resultant force can also be expressed as the rate of change of momentum, i.e., $\dot{\mathbf{p}}$.

$$\dot{\mathbf{p}} = -\gamma \left(\dot{\mathbf{q}} - \beta \begin{bmatrix} -\frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \end{bmatrix} \mathbf{q} \right) - \mathbf{u}$$

We can also express $\dot{\mathbf{q}}$ in terms of $\mathbf{p}$. A scale factor of $\frac{1}{m}$ needs to be applied since momentum is a function of mass.

$$\dot{\mathbf{p}} = -\gamma \left(\frac{\mathbf{p}}{m} \right) + \gamma\beta \begin{bmatrix} -\frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \end{bmatrix} \mathbf{q} - \mathbf{u}$$
$$\dot{\mathbf{q}} = \frac{\mathbf{p}}{m}$$

It is now possible to convert this to state-space:

$$\begin{bmatrix} \dot{q}_x \\ \dot{q}_y \\ \dot{p}_x \\ \dot{p}_y \end{bmatrix} = \begin{bmatrix} 0 & 0 & \frac{1}{m} & 0 \\ 0 & 0 & 0 & \frac{1}{m} \\ -\frac{\gamma\beta\sqrt{2}}{2} & -\frac{\gamma\beta\sqrt{2}}{2} & -\frac{\gamma}{m} & 0 \\ \frac{\gamma\beta\sqrt{2}}{2} & -\frac{\gamma\beta\sqrt{2}}{2} & 0 & -\frac{\gamma}{m} \end{bmatrix} \begin{bmatrix} q_x \\ q_y \\ p_x \\ p_y \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_x \\ u_y \end{bmatrix}$$

2 Question 2

To implement the implicit Euler method for solving this state-space ODE, the following equation can be used:

$$X(t_i) = (I - \Delta t_i A(t_i))^{-1} (X(t_{i-1}) + \Delta t_i B(t_i))$$

Note that $X = \begin{bmatrix} q_x \\ q_y \\ p_x \\ p_y \end{bmatrix}$.

```
[16]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse
import random
import scipy as sp
from matplotlib.collections import LineCollection

random.seed(2110409)
plt.rcParams['figure.figsize'] = [10, 5]

m = 1.5
gamma = 2.5
omega = 2

A = np.array([[0, 0, 1/m, 0],
               [0, 0, 0, 1/m],
               [((-gamma*omega*2**0.5)/2), ((-gamma*omega*2**0.5)/2), -gamma/m, 0],
               [((gamma*omega*2**0.5)/2), ((-gamma*omega*2**0.5)/2), 0, -gamma/m]])

B = np.array([[0, 0], [0, 0], [1, 0], [0, 1]])
uEuler = 0*np.array([1, 1])

x0 = np.array([5.0, 5.0, 0.0, 0.0])
dt = 0.001
T = np.arange(0.0, 30.0 + dt, dt)

X = np.zeros((4, len(T)))
X[:, 0] = x0

for i in range(len(T))[1:]:
    X[:, i] = (np.linalg.solve(np.eye(4) - dt*A, X[:, i-1] + dt*B@uEuler))

fig, ax = plt.subplots()
points = np.array([X[0, :], X[1, :]]).T.reshape(-1, 1, 2)
segments = np.concatenate([points[:-1], points[1:]], axis=1)
normT = plt.Normalize(T.min(), T.max())
lC = LineCollection(segments, cmap="gist_earth", norm=normT)
```

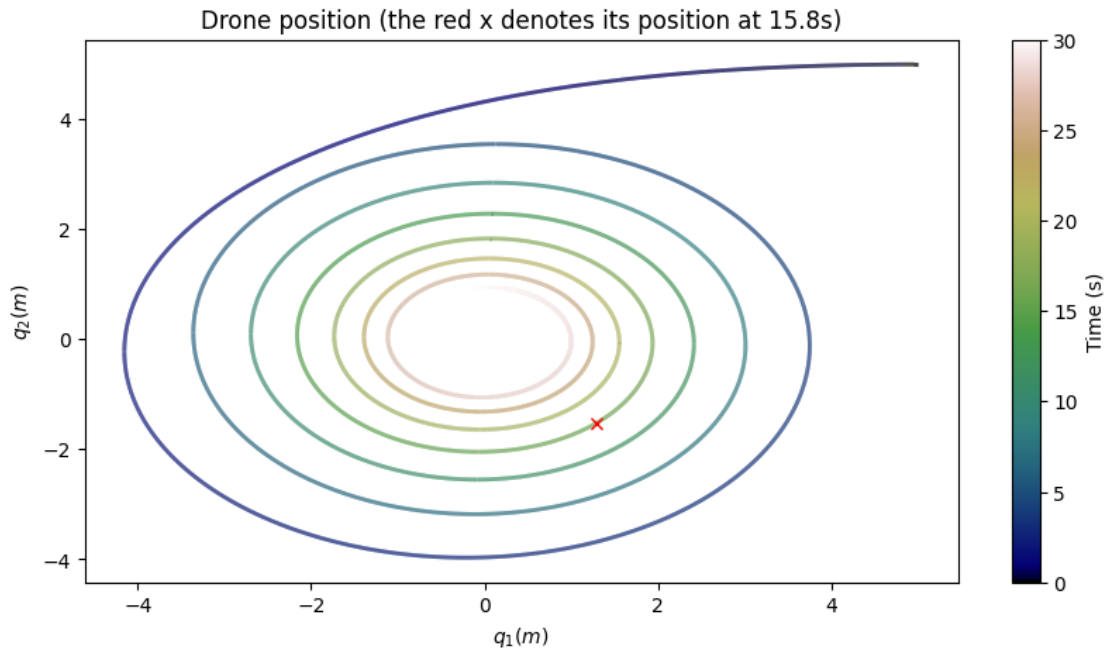
```

lC.set_array(T)
lC.set_linewidth(2)
line = ax.add_collection(lC)
fig.colorbar(line, ax=ax, label="Time (s)")

ax.plot(X[0, T == 15.8], X[1, T == 15.8], color="red", marker = "x")
ax.set_title("Drone position (the red x denotes its position at 15.8s)"); ax.
    ↪set_xlabel("$q_{1}$ (m)$"); ax.set_ylabel("$q_{2}$ (m)$");
dist = (X[0, T == 15.8]**2 + X[1, T == 15.8]**2)**0.5
print("Question 1: The magnitude of the distance between the drone and the_
    ↪origin is %.3f metres at 15.8 seconds." %dist[0])

```

Question 1: The magnitude of the distance between the drone and the origin is 1.995 metres at 15.8 seconds.



After plotting the two positions against each other, we can verify that the drone follows a spiral path towards the origin. It also drifts within two metres of the origin at around 15.8 seconds. This distance is computed as the distance between two points

3 Question 3

The Kalman filter relies on five equations, all of which will be detailed below.

$$\hat{x}_i = F_d \hat{x}_{i-1} + B_d u_i \quad (1)$$

$\hat{x}$ is the estimate, F_d is the state transition matrix, and B_d is the control matrix.

F_i and B_i are the discretised matrices of A and B from the state-space model. To calculate F_d , we can use the matrix exponential $e^{\Delta t A}$. To calculate B_d , we can use the formula $(F_d - I_4) F_d^{-1} B$.

$$P_i = F_d P_{i-1} F_d' + Q \quad (2)$$

P_i is the state covariance matrix and Q is the process noise. However, in our model this will be 0. (Note that the observation noise R is added at each integer time to the true states obtained earlier.)

In the update step, there are three equations:

$$K = P_i H' (H P_i H' + R)^{-1} \quad (3)$$

K is the Kalman gain, and the observation matrix H will be:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

We are only observing the position values and not the momentum. This matrix would simply take only the position states from a state vector and not the momentum states. Finally, the last two equations are:

$$\hat{x}_i = \hat{x}_i + K' (y_i - H \hat{x}_i) \quad (4)$$

$$P_i = (I_4 - KH) P_i \quad (5)$$

```
[17]: tspan = np.arange(0.0, 30 + 1, 1)
H = np.array([[1, 0, 0, 0], [0, 1, 0, 0]]) # observation matrix
Q = np.eye(4)*0 # process noise
R = np.eye(2)*0.5**2 # observation noise

y = np.zeros((2, 31))
q_true = X[:, np.round(T) == T]
# q_true[:,0] = q_true[:,0] + np.random.multivariate_normal(np.zeros(4), Q).T
y[:,0] = H@q_true[:,0] + np.random.multivariate_normal(np.zeros(2), R).T

for i in range(len(tspan))[1:]:
    # q_true[:,i] = q_true[:,i] + np.random.multivariate_normal(np.zeros(4), Q).
    ↪ T
    y[:,i] = H@q_true[:,i] + np.random.multivariate_normal(np.zeros(2), R).T

q_hat_0 = 0*np.array([1, 1, 0, 0])
q_hat = np.zeros((4, 31))
q_hat[:,0] = q_hat_0
P = np.zeros((31, 4, 4))
P[0] = np.eye(4)*1e6
```

```

Fi = sp.linalg.expm(1*A) # discretisation of A using the time step
Bi = (Fi-np.eye(4))@np.linalg.inv(A)@B # discretisation of B using Fi
uKalman = 0*np.array([1, 1])

for i in range(len(tspan))[1:]:
    q_hat[:,i] = Fi@q_hat[:,i-1] + Bi@uKalman
    P[i] = Fi@P[i-1]@Fi.T + Q

    K = P[i]@H.T@np.linalg.inv(H@P[i]@H.T + R)
    q_hat[:,i] = q_hat[:,i] + K@(y[:,i] - H@q_hat[:,i])
    P[i] = (np.eye(4)-K@H)@P[i]

fig1, ax1 = plt.subplots()
ax1.plot(q_true[0,:], q_true[1:], ":", color="black", label="True States")
ax1.scatter(y[0,:], y[1:], color="orange", label="Observations")
ax1.plot(q_hat[0,:], q_hat[1:], color="green", label="Estimated States")

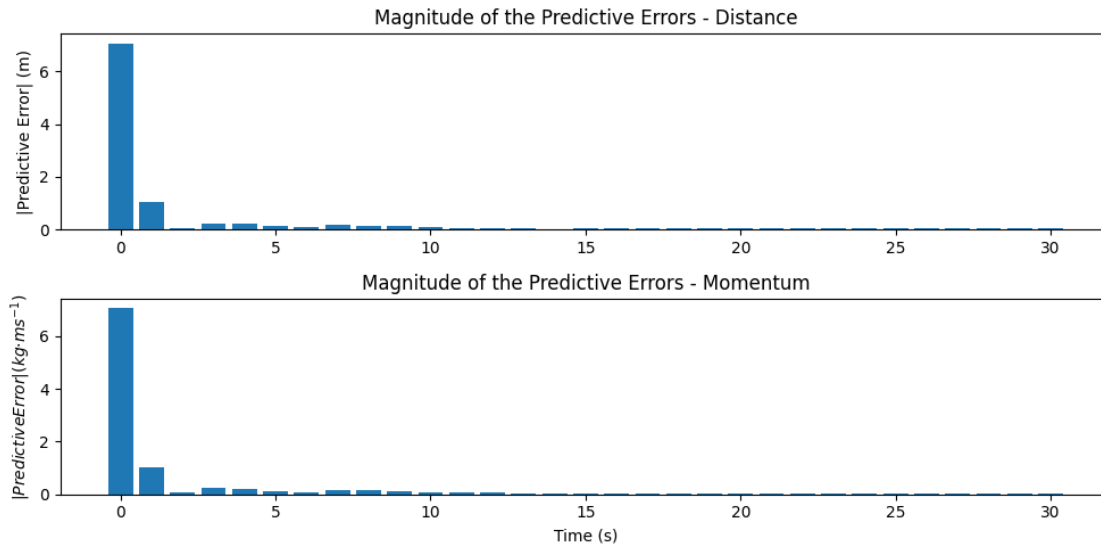
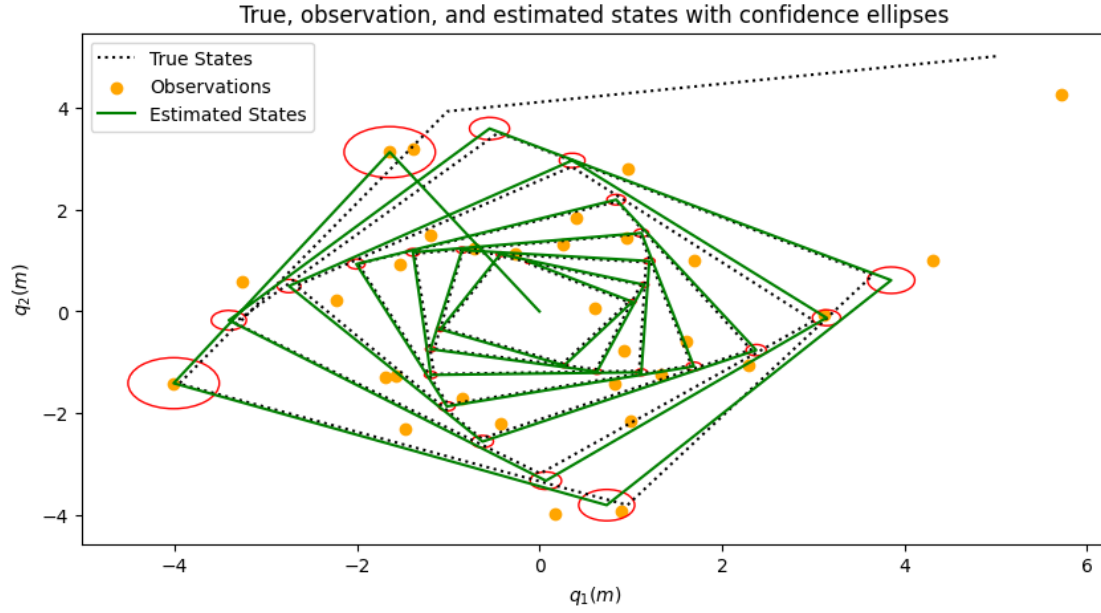
PPos = P[:, :2, :2]
for i in range(len(tspan))[1:]: # the first one is excluded since it would
    ↪heavily distort the axes
    mean = q_hat[0:2,i]
    eigVals, eigVecs = np.linalg.eig(PPos[i,:,:])
    angle = np.degrees(np.arctan2(eigVecs[1, 0], eigVecs[0, 0]))
    confidenceRegion = Ellipse(xy=mean, width=2*np.sqrt(eigVals[0]),
    ↪height=2*np.sqrt(eigVals[1]), angle=angle, edgecolor="r", facecolor="none")
    ax1.add_patch(confidenceRegion)

ax1.legend()
ax1.set_title("True, observation, and estimated states with confidence
    ↪ellipses"); ax1.set_xlabel("$q_{1}$ (m)$"); ax1.set_ylabel("$q_{2}$ (m)$");

fig2, (ax21, ax22) = plt.subplots(nrows=2, ncols=1)
predictiveErrorDist = q_hat[[0, 1],:]-q_true[[0, 1],:]
magspEDist = np.linalg.norm(predictiveErrorDist, axis=0)
ax21.bar(tspan, magspEDist)
ax21.set_title("Magnitude of the Predictive Errors - Distance")
ax21.set_ylabel("|Predictive Error| (m)")

predictiveErrorMom = q_hat[[2, 3],:]-q_true[[2, 3],:]
magspEMom = np.linalg.norm(predictiveErrorMom, axis=0)
ax22.bar(tspan, magspEDist)
ax22.set_title("Magnitude of the Predictive Errors - Momentum")
ax22.set_xlabel("Time (s)"); ax22.set_ylabel("$|Predictive Error|
    ↪(kg ms^{-1})$");
fig2.tight_layout()

```



Note that in the first figure, the confidence ellipse has been ignored as it would heavily distort the axes. The Kalman filter performs well, with the estimated states closely tracking the true states, even though our initial predicted state was incorrect.

The confidence ellipses start off large, however as time pregresses the Kalman filter gets more confident and the regions become smaller. We can see this also with the predictive error of the system, where the error is initially large but decreases significantly.

4 Question 4

To implement a PD control mechanism for the drone using the estimated states (position only), the error (difference between the desired position and the current estimated position) needs to be considered. The proportional control is a simple scale factor K_P of the error. The derivative control looks at the derivative of the error with respect to time. This can be scaled with another value, K_D .

After some testing, a value of 500 was chosen for K_P and 0.1 for K_D .

```
[18]: def implicitEuler(qInit, uInput, Amat, Bmat):
    dt = 0.001
    trueStates = np.zeros((4,2))
    trueStates[:,0] = qInit
    trueStates[:,1] = (np.linalg.solve(np.eye(4) - dt*Amat, trueStates[:,0] +
    dt*Bmat*uInput))
    return trueStates

def kalmanFilter(Amat, Bmat):
    dt = 0.001
    Time = np.arange(0.0, 30.0 + dt, dt)
    H = np.array([[1, 0, 0, 0], [0, 1, 0, 0]])
    R = np.eye(2)*0.5**2

    qTrue = np.zeros((4, len(Time)))
    qTrue[:,0] = np.array([5.0, 5.0, 0.0, 0.0])
    obs = np.zeros((2, len(Time)))
    obs[:,0] = H@qTrue[:,0] + np.random.multivariate_normal(np.zeros(2), R).T

    qPredicted = np.zeros((4, len(Time)))
    qInit = 0*np.array([1, 1, 0, 0])
    qPredicted[:,0] = qInit

    P = np.zeros((len(Time), 4, 4))
    P[0] = np.eye(4)*1e6
    Fi = sp.linalg.expm(dt*Amat)
    Bi = (Fi-np.eye(4))@np.linalg.inv(Amat)*Bmat

    qStar = np.array([-1, 1])
    err = np.zeros((2, len(Time)))
    err[:,0] = qStar-qInit[1:2]
    uInput = np.zeros((2, len(Time)))

    for i in range(len(Time))[1:]:
        eulerStates = implicitEuler(qTrue[:,i-1], uInput[:,i-1], Amat, Bmat)
        qTrue[:,i] = eulerStates[:,-1]
        obs[:,i] = H@qTrue[:,i] + np.random.multivariate_normal(np.zeros(2), R).T
```

```

    qPredicted[:,i] = Fi@qPredicted[:,i-1] + Bi@uInput[:,i-1]
    uInput[:,i], err[:,i] = PD_Controller(qStar, qPredicted[[0, 1], i], u
    err[:, i-1]) # control relies only on estimate
    P[i] = Fi@P[i-1]@Fi.T

    if round(Time[i]) == Time[i]:
        K = P[i]@H.T@np.linalg.inv(H@P[i]@H.T + R)
        qPredicted[:,i] = qPredicted[:,i] + K@(obs[:,i] - H@qPredicted[:,i])
        P[i] = (np.eye(4)-K@H)@P[i]

    return qPredicted, qTrue, obs, err, uInput

def PD_Controller(finalPos, estimatedPos, prevError):
    error = finalPos-estimatedPos
    derivative = (error-prevError)/1e-3
    KpX = 500; KpY = 500
    KdX = 0.1; KdY = 0.1

    uControl = np.zeros((2,))
    uControl[0] = KpX*error[0] + KdX*derivative[0]
    uControl[1] = KpY*error[1] + KdY*derivative[1]
    uMagnitude = (uControl[0]**2 + uControl[1]**2)**0.5
    if uMagnitude >= 5:
        scale = 5/uMagnitude
        uControl[0] = uControl[0]*scale
        uControl[1] = uControl[1]*scale

    return uControl, error

qHat, qTrue = kalmanFilter(A,B)[0:2]
fig3, ax3 = plt.subplots()
ax3.plot(qHat[0,:], qHat[1:], color="black", label="Truth")
ax3.plot(qTrue[0,:], qTrue[1:], color="orange", label="Estimate")
ax3.legend()
ax3.set_title("PD Control Applied to the Drone")
ax3.set_xlabel("$q_{1}$ (m)$"); ax3.set_ylabel("$q_{2}$ (m)$")

fig4, (ax41, ax42) = plt.subplots(nrows=2, ncols=1)
ax41.plot(T, qHat[2,:], label="$p_{1}$")
ax41.plot(T, qHat[3,:], label="$p_{2}$")
ax41.set_title("Estimated Momentum of the Drone")
ax41.legend()
ax41.set_xlabel("Time (s)"); ax41.set_ylabel("$Momentum (kg ms^{-1})$");

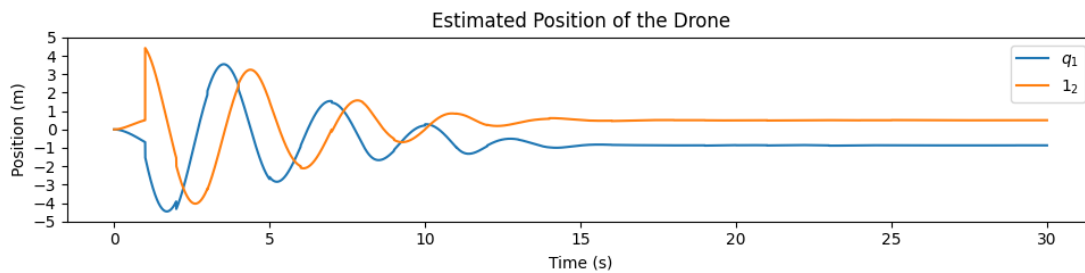
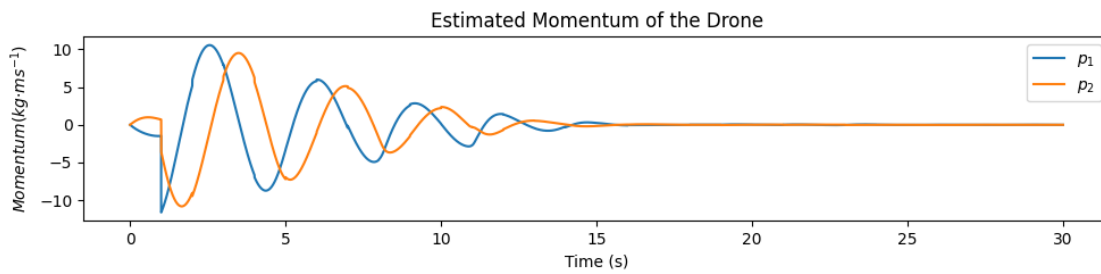
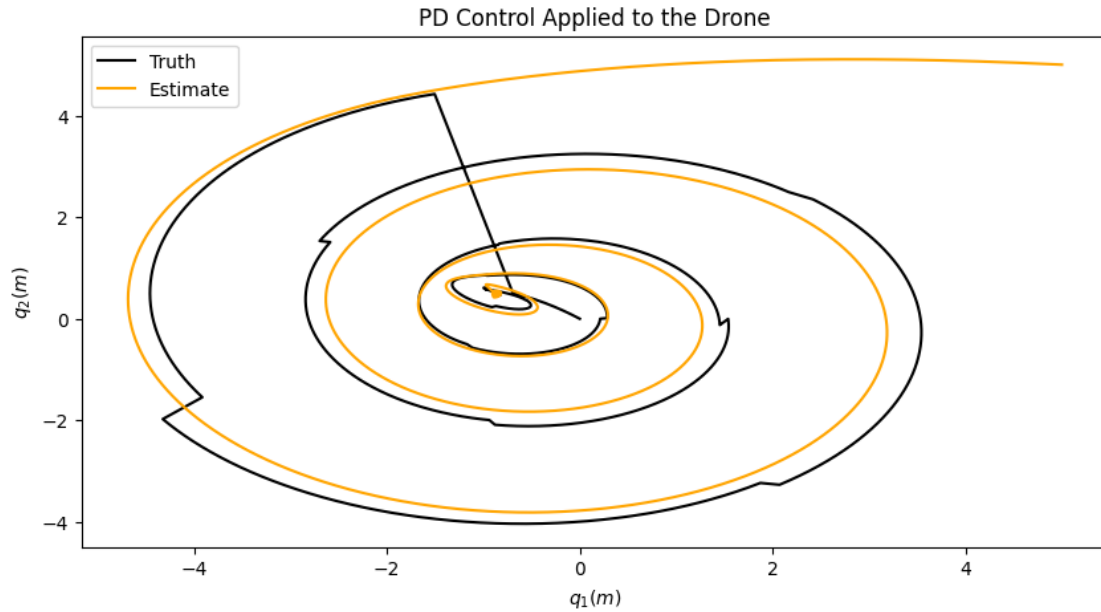
ax42.plot(T, qHat[0,:], label="$q_{1}$")
ax42.plot(T, qHat[1,:], label="$q_{2}$")

```

```

ax42.set_title("Estimated Position of the Drone")
ax42.legend()
ax42.set_xlabel("Time (s)"); ax42.set_ylabel("Position (m)");
yticks5 = list(range(-5, 6))
ax42.set_yticks(yticks5);
fig4.tight_layout()

```



The first plot is made to verify that the estimated states follow the true states as expected, which

they do. To check whether the drone comes to rest, its momentum can be plotted. Since the mass is constant, we can tell that its velocity goes to 0 at around halfway through the simulation, indicating that it has come to rest.

At around the same time it comes to rest, we can see that the position of the drone has reached (within an acceptable tolerance) the desired position. Since the force constraint of 5N is quite low, the accuracy of the desired position must be given a larger than usual tolerance (± 0.5). If the force limit is increased to around 100N, the drone can be brought to rest near the desired position within 1 decimal point accuracy.